Ingegneria del Software – 17/06/2025 – Soluzione

Consegnare SOLO i fogli del tema.	**Attaccare l'etichetta in questa zona.**
Non separarli. Altri fogli saranno ignorati. I fogli sono fronte/retro. 2 ore di tempo. Si possono consultare libri e appunti. Tutti i dispositivi elettronici sono vietati.	Compiti senza etichetta non sono validi.

Esercizio 1

Si consideri la classe Java `StudyCenter` per gestire la prenotazione di aule studio universitarie.

```
public class StudyCenter {
  // Restituisce l'insieme delle aule registrate nel sistema.
  public /*@ pure @*/ Set<Room> getAllRooms();

  // Restituisce la lista delle aule che soddisfano i seguenti criteri:
  // (i) hanno capacita' per almeno people persone;
  // (ii) sono disponibili nell'intervallo timeInterval.
  // La lista e' ordinata in ordine alfabetico per nome.
  public /*@ pure @*/ List<Room> availableRooms(int people, TimeInterval timeInterval);

  // Effettua una prenotazione dell'aula specificata per l'intervallo indicato.
  // Solleva UnavailableException se l'aula non e' libera in quell'intervallo.
  public void reserve(Room room, TimeInterval timeInterval) throws UnavailableException;
}

public class Room {
  // Ritorna la capienza massima dell'aula.
  public /*@ pure @*/ int getCapacity();

  // Ritorna il nome identificativo dell'aula.
  public /*@ pure @*/ String getName();

  // Ritorna true sse l'aula e' disponibile per tutto l'intervallo specificato.
  public /*@ pure @*/ boolean isFree(TimeInterval interval);

  // Prenota l'aula per l'intervallo specificato.
  // Richiede che l'aula sia libera per tutto l'intervallo.
  public void book(TimeInterval interval);
}

public class TimeInterval {
  // Ritorna il giorno della prenotazione
  public /*@ pure @*/ Day getDay();

  // Ritorna l'orario di inizio (in minuti da mezzanotte, ad esempio 9:00 = 540)
  public /*@ pure @*/ int getStart();

  // Ritorna l'orario di fine (escluso, in minuti da mezzanotte, es. 10:00 = 600)
  public /*@ pure @*/ int getEnd();

  // Ritorna true sse l'intervallo this si sovrappone a other
  public /*@ pure @*/ boolean overlaps(TimeInterval other);
}
```

La classe Day non è ulteriormente specificata.

Domanda a) Si specifichi in JML il metodo `availableRooms()`.

Soluzione

```
//@ requires people >= 0 && timeInterval != null;
//@ ensures
//@ (\forall Room r; ; \result.contains(r) <==> getAllRooms().contains(r) &&
//@    r.getCapacity() >= people && r.isFree(timeInterval) )
//@ (* La lista non contiene duplicati *)
//@ (\forall int i; 0 <= i && i < \result.size()-1;
//@   (\forall int j; 0 <= j && j < i; !\result.get(i).equals(\result.get(j))) )
//@ (* Gli elementi sono ordinati alfabeticamente per nome *)
//@ (\forall int i; 0 <= i && i< \result.size()-1;
//@   \result.get(i).getName().compareTo(\result.get(i+1).getName()) <= 0));
public /*@ pure @*/ List<Room> availableRooms(int people, TimeInterval timeInterval);
```

Domanda b) Si specifichi in JML il metodo `reserve()`.

Soluzione

```
//@ requires room != null && timeInterval != null && getAllRooms().contains(room)
//@ ensures
//@ (* L'aula era disponibile al momento della chiamata*)
//@ \old(availableRooms(room.getCapacity(), timeInterval).contains(room)) &&
//@ (* L'insieme delle aule non cambia *)
//@ (\forall Room r; ; (\old(getsAllRooms.contains(r)) <==> getsAllRooms.contains(r))) &&
//@ (* La disponibilita' delle altre aule non cambia *)
//@ (\forall Room r; !r.equals(room);
//@   (\forall int p; p>0; (\forall TimeInterval t; ;
//@     availableRooms(p, t).contains(r) <==> \old(availableRooms(p, t)) ) ) ) &&
//@ (* La disponibilita' di room cambia per tutti e soli gli intervalli che si sovrappongono a
//@    timeInterval e per tutti i numeri di occupanti *)
//@ (\forall TimeInterval t; ;
//@    t.overlaps(timeInterval) ==> \forall int p; p>0; !availableRooms(p, t).contains(room) &&
//@    !t.overlaps(timeInterval) ==> \forall int p; p>0; availableRooms(p, t).contains(room) <==>
//@       \old(availableRooms(p, t).contains(room) ) ) &&
//@ (* Se l'aula e' occupata, viene sollevata un'eccezione *)
//@ signals (UnavailableException e)
//@ !\old(availableRooms(people, timeInterval).contains(room)) &&
//@ (\forall Room r; ; (\old(getsAllRooms.contains(r)) <==> getsAllRooms.contains(r))) &&
//@ (\forall int p; p > 0;
//@   (\forall TimeInterval t; ;
//@     availableRooms(p, t).equals(\old(availableRooms(p, t) ) ) ) )
public void reserve(Room room, TimeInterval timeInterval) throws UnavailableException;
```

Domanda c) Si consideri un'implementazione che utilizza una `Map` per memorizzare le aule e gli intervalli prenotati, come riportato di seguito. Per tale implementazione si specifichi un invariante di rappresentazione e una funzione di astrazione.

```
public class StudyCenter {
  // Mappa ogni stanza al suo insieme di intervalli prenotati
  private Map<Room, Set<TimeInterval>> bookings;
  ...
}
```

Soluzione

Invariante di rappresentazione (RI).

```
//@ private invariant bookings != null &&
//@ (\forall Room r; bookings.keySet().contains(r); r != null) &&
//@ (\forall Room r; bookings.containsKey(r);
//@   bookings.get(r) != null &&
//@   (\forall TimeInterval t1; bookings.get(r).contains(t1);
//@       t1 != null && (\forall TimeInterval t2;
//@          bookings.get(r).contains(t2) && t1 != t2; !t1.overlaps(t2)) );
```

Funzione di astrazione (FA).

```
//@ private invariant
//@ (\forall Room r; ; bookings.containsKey(r) <==> getAllRooms().contains(r)) &&
//@ (\forall Room r; ;
//@   (\forall TimeInterval t; ; (\forall int p; ;
//@     availableRooms(p, t).contains(r) <==> (
//@       bookings.containsKey(r) && p <= r.getCapacity() &&
//@       !(\exists TimeInterval t1; bookings.get(r).contains(t1); t.overlaps(t1))
//@ ) ) )
```

Domanda d) Si vuole estendere il sistema introducendo un nuovo tipo di aula: `TemporaryRoom`, che può essere prenotata una sola volta al giorno. La proposta è di implementare questa logica ridefinendo il metodo `book(TimeInterval)` nella sottoclasse `TemporaryRoom`.

`TemporaryRoom` può essere considerata un sottotipo corretto di Room secondo il principio di sostituzione di Liskov (LSP)? Motivare la risposta.

Soluzione No, TemporaryRoom non può essere considerata un sottotipo corretto di `Room` secondo il principio di sostituzione di Liskov. Il contratto della classe `Room` prevede che, se un'aula è libera in un certo intervallo, allora possa essere prenotata tramite `book()` (independentemente dall'essere stata già prenotata nello stesso giorno). La sottoclasse `TemporaryRoom`, invece, introduce un vincolo più restrittivo: una volta prenotata in un certo giorno, la stanza non può più essere prenotata in quel giorno, anche se resa nuovamente libera. Questo indebolisce la postcondizione di `book()`.

Esercizio 2

Si consideri la seguente classe `Garden` che permette di gestire le condizioni climatiche e l'impianto di irrigazione di un giardino.

```
public class Garden {
  private double temp, humidity;
  private Object lockT, lockH;

  public Garden() {
    lockT = new Object(); lockH = new Object();
  }
  public double getTemp() {
    synchronized(lockT) { return temp; }
  }
  public void setTemp(double t) {
    synchronized(lockT) { temp = t; }
  }
  public double getHumidity() {
    synchronized(lockH) { return humidity; }
  }
  public void setHumidity(double h) {
    synchronized(lockH) { humidity = h; }
  }

  private void activateSprinklers() {
    // code to activate sprinklers and water the garden
  }
}
```

Domanda a) La classe è correttamente sincronizzata? Motivare la propria risposta e in caso di risposta negativa spiegare cosa andrebbe cambiato per evitare conflitti nell'accesso a memoria condivisa da parte di thread diversi.

Soluzione

La classe è correttamente sincronizzata.

Domanda b) Si aggiunga alla classe `Garden` un metodo `public void waterIfCondMet()` (opportunamente sincronizzato) che invoca il metodo privato `activateSprinklers` in un thread separato ma solo se l'umidità è inferiore a 0.95 e la temperatura è inferiore a 35.

Soluzione

Il metodo deve accedere ai due attributi privati. Lo farà acquisendo entrambi i lock in cascata.

```
public void waterIfCondMet() {
  synchronized(lockT) {
    synchronized(lockH) {
      if (humidity < 0.95 && temp < 35) {
        new Thread(() -> activateSprinklers()).start();
      }
    }
  }
}
```

Domanda c) Si aggiunga alla classe `Garden` un metodo `public void waitForCond()` che sospende il chiamante se le condizioni per l'irrigazione (umidità minore di 0.95 e temperatura minore di 35) non sono raggiunte. Si indichi se e quali altri metodi debbano essere modificati e come.

Soluzione

In questo caso è necessario non solo accedere "atomicamente" ai due attributi `humidity` e `temp` ma anche sospendere il thread chiamante in attesa del verificarsi della condizione utile a proseguire. In questi casi non è possibile acquisire i due lock (perché `wait` rilascerebbe uno solo dei due lock, mettendosi in una condizione di potenziale deadlock o starvation). L'unica soluzione è quindi ripensare tutta la sincronizzazione della classe, ad esempio sincronizzando tutti i metodi su `this`, come nella seguente implementazione (si noti l'aggiunta di `notifyAll()` nei due "setter"):

```
public class GardenSync {
  private double temp, humidity;

  public synchronized double getTemp() {
    return temp;
  }
  public synchronized void setTemp(double t) {
    temp = t;
    notifyAll();
```

```
  }
  public synchronized double getHumidity() {
    return humidity;
  }
  public synchronized void setHumidity(double h) {
    humidity = h;
    notifyAll();
  }
  public synchronized void waitForCond() throws InterruptedException {
    while(humidity>=0.95 || temp>=35) wait();
  }
  private void activateSprinklers() {
    // code to activate sprinklers and water the garden
  }
}
```

Esercizio 3

Si considerino le seguenti classi Java:

```java
abstract class Employee {
  protected String name;

  public Employee(String name) { this.name = name; }
  public abstract String getRole();
  public void printInfo() { System.out.println(name + " is a " + getRole()); }
}

abstract class Developer extends Employee {
  public Developer(String name) { super(name); }
  public String getRole() { return "Developer"; }
  public abstract void writeCode();
}

class BackendDeveloper extends Developer {
  public BackendDeveloper(String name) { super(name); }
  public void writeCode() { System.out.println(name + " writes backend code"); }
}

class FrontendDeveloper extends Developer {
  public FrontendDeveloper(String name) { super(name); }
  public void writeCode() { System.out.println(name + " writes frontend code"); }
  public String getRole() { return "Frontend Developer"; }
}

class Manager extends Employee {
  public Manager(String name) { super(name); }
  public String getRole() { return "Manager"; }
}
```

e il seguente frammento di codice le cui righe sono state numerate per riferimento:

```
1  Employee e1 = new BackendDeveloper("Alice");
2  Employee e2 = new FrontendDeveloper("Bob");
3  Developer d1 = new FrontendDeveloper("Clara");
4  Employee e3 = new Manager("Dan");
5  FrontendDeveloper fd = (FrontendDeveloper) e2;
6  BackendDeveloper bd;
7  e1.printInfo();
8  e2.printInfo();
9  d1.printInfo();
10 d1.writeCode();
11 fd.writeCode();
12 e3.printInfo();
13 bd = (BackendDeveloper) e2;
14 bd.writeCode();
```

Domanda a) Indicare le istruzioni che generano errore in compilazione o run-time, specificando il tipo di errore e la motivazione.

Soluzione Riga 13 genera errore a run-time: `ClassCastException` poiché e2 è un `FrontendDeveloper`, e non può essere castato a `BackendDeveloper`. In conseguenza dell'errore precedente non va considerata neppure la riga 14.

Domanda b) Avendo eliminato le righe con errore, scrivere cosa viene stampato per ogni riga rimanente.

Soluzione

Alice is a Developer
Bob is a Frontend Developer
Clara is a Frontend Developer
Clara writes frontend code
Bob writes frontend code
Dan is a Manager

Esercizio 4

Si consideri il seguente metodo:

```
1  public static int sumEvenPositives(int n, int[] x) {
2    if (x == null || x.length < n) {
3      return -1;
4    }
5    int sum = 0;
6    for (int i = 0; i < n; i++) {
7      if ((x[i] % 2 == 0) && (x[i] > 0 || x[i] == -2)) {
8        sum += x[i];
9      }
10   }
11   return sum;
12 }
```

Si risponda alle seguenti domande, giustificando (sinteticamente) la risposta.

Domanda a) Si determini un insieme *minimo* di test per garantire la copertura delle istruzioni (statement coverage).

Soluzione Bastano due casi, per esempio `x=null` (per eseguire la linea 3) e `(n=1, x=[2])` per entrare nel loop e nell'`if`, in modo da eseguire anche la linea 8.

Domanda b) Si determini un insieme *minimo* di test per garantire la copertura delle decisioni (edge coverage).

Soluzione

Occorre gestire il ramo "nascosto" dell'if. Bastano ancora due casi, `x=null` e `(n=2, x=[2,1])`, in modo che alla seconda iterazione del loop la condizione dell'`if` risulti falsa.

Domanda c) Si determini un insieme *minimo* di test per garantire la copertura delle decisioni e delle condizioni (edge and condition coverage).

Soluzione

Per coprire le condizioni dell'`if` alla riga 2, occorre aggiungere, al caso `x=null`, anche il caso in cui `n` è più grande della lunghezza dell'array: per esempio `(n=2, x=[0])`.

Per coprire le condizioni dell'`if` di riga 7: il caso precedente copriva il caso `true` e `false` di `(x[i] % 2 == 0)` e il caso `true` di `x[i]>0`. Se modifichiamo `x` in modo che contenga anche il valore `-2`, abbiamo un test che copre il caso `true` di `x[i] > 0` e il caso `true` di `x[i] == -2`. Manca un ultimo caso, per esempio `x[i]=-4`, per coprire anche il caso `false` di `x[i] == -2`.

In conclusione, bastano 3 casi di test, per esempio: 1) `(x=null, n=[])`, 2) `(n=2, x=[0])`, 3) `(n=4, x=[2, 1, -2, -4])`.